

Node.js Basics (1–8)

❏ What is Node.js?

Node.js is a **JavaScript runtime built on Chrome's V8 engine** .

It allows JavaScript to run outside the browser, mainly for backend development.

Key features:

- Event-driven
 - Non-blocking I/O
 - Single-threaded (with event loop)
-

❏ How does Node.js work internally?

Node.js architecture:

- V8 Engine → Executes JS
- Event Loop → Handles async tasks
- Libuv → Manages I/O, threads
- Thread pool → For file system, DNS, crypto

Flow:

1. Request comes
 2. Event loop processes
 3. Async tasks delegated to thread pool
 4. Callback executed
-

❏ What is the Event Loop?

The event loop handles asynchronous operations.

Phases:

- Timers
- Pending callbacks
- Idle
- Poll
- Check

- Close callbacks

Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

console.log("End");
```

Output:

```
Start
End
Timeout
```

Because setTimeout runs in timers phase.

4 What is non-blocking I/O?

Node does not wait for I/O operations to complete.

Example:

```
const fs = require("fs");

fs.readFile("file.txt", "utf8", (err, data) => {
  console.log(data);
});

console.log("Done");
```

"Done" prints first.

5 What is callback hell?

Nested callbacks causing unreadable code.

Example:

```
getUser(id, (user) => {
  getOrders(user.id, (orders) => {
    getPayment(orders[0], (payment) => {
      console.log(payment);
    });
  });
});
```

Solution:

- Promises
 - Async/Await
-

❏ What are Promises?

Represents eventual completion of async operation.

```
const promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});  
  
promise.then(console.log);
```

❏ What is async/await?

Syntactic sugar over promises.

```
async function fetchData() {  
  const data = await getData();  
  console.log(data);  
}
```

Cleaner than chaining `.then()`.

❏ What is process object?

Provides information about current Node process.

```
console.log(process.env.PORT);
```

Used for environment variables.

❏ Asynchronous & Advanced Concepts (9–18)

❏ What is setImmediate vs setTimeout?

- setTimeout → timers phase
- setImmediate → check phase

Execution order depends on context.

What is nextTick?

Runs before next event loop iteration.

```
process.nextTick(() => {  
  console.log("NextTick");  
});
```

High priority.

11 What is the difference between synchronous and asynchronous functions?

Sync blocks execution.
Async does not block.

12 What is stream in Node.js?

Used for processing large data chunks.

Types:

- Readable
- Writable
- Duplex
- Transform

Example:

```
const fs = require("fs");  
const readStream = fs.createReadStream("file.txt");  
  
readStream.on("data", chunk => {  
  console.log(chunk);  
});
```

13 Why use streams?

- Memory efficient
 - Handles large files
 - Faster processing
-

14 What is buffer?

Raw binary data storage.

```
const buf = Buffer.from("Hello");
console.log(buf);
```

Used in networking and file handling.

15 What is middleware?

Function executed during request lifecycle (Express).

```
app.use((req, res, next) => {
  console.log("Request received");
  next();
});
```

16 What is clustering?

Uses multiple CPU cores.

```
const cluster = require("cluster");
```

Improves scalability.

17 What is child_process?

Executes external commands.

```
const { exec } = require("child_process");
exec("ls", (err, stdout) => {
  console.log(stdout);
});
```

18 What is worker_threads?

Used for CPU-bound tasks.

Better than cluster for shared memory.

3 Express & API Development (19–28)

19 What is Express.js?

Minimal web framework for Node.

```
const express = require("express");
const app = express();
```

```
app.get("/", (req, res) => {
  res.send("Hello");
});
```

20 What is routing?

```
app.post("/login", (req, res) => {});
```

21 How to handle request body?

```
app.use(express.json());
```

22 What is error handling middleware?

```
app.use((err, req, res, next) => {
  res.status(500).send("Error");
});
```

23 What is JWT authentication?

```
const jwt = require("jsonwebtoken");

const token = jwt.sign({ id: 1 }, "secret");
```

Used for stateless authentication.

24 What is CORS?

Allows cross-origin requests.

```
const cors = require("cors");
```

```
app.use(cors());
```

25 What is rate limiting?

Prevent API abuse.

26 How to handle file uploads?

Use multer.

27 How to validate request?

Use express-validator or Joi.

28 How to connect to PostgreSQL?

```
const { Pool } = require("pg");
```

```
const pool = new Pool({  
  user: "postgres",  
  password: "password",  
});
```

⚡ 4 Performance & Scaling (29–38)

29 How to scale Node.js app?

- Cluster mode
 - Load balancer
 - Caching (Redis)
 - Horizontal scaling
-

30 What is memory leak in Node?

Unreleased memory references.

Example:

- Global variables
- Unremoved event listeners

31❑ How to debug memory leaks?

Use:

- heapdump
 - Chrome DevTools
-

32❑ What is event emitter?

```
const EventEmitter = require("events");
```

Used for custom events.

33❑ What is backpressure?

When writable stream slower than readable.

Handle with pipe().

34❑ What is PM2?

Process manager for Node apps.

35❑ How to secure Node app?

- Use HTTPS
 - Helmet
 - Rate limit
 - Validate inputs
-

36❑ What is Nginx used for?

Reverse proxy and load balancer.

37❑ What is connection pooling?

Reuse DB connections.

38❑ What is horizontal vs vertical scaling?

Horizontal → more servers

Vertical → bigger server

5 Advanced Topics (39–50)

39 What is microservices architecture?

Breaking app into small services.

40 What is REST vs GraphQL?

REST → multiple endpoints

GraphQL → single flexible endpoint

41 What is WebSocket?

Real-time communication.

```
const io = require("socket.io")(server);
```

42 How to handle background jobs?

Use:

- Bull
 - RabbitMQ
 - Redis queue
-

43 What is dotenv?

Loads environment variables.

44 What is dependency injection in Node?

Using libraries like Inversify.

45 What is ORM?

Sequelize, Prisma.

46 ☐ What is schema validation?

Define expected request structure.

47 ☐ What is throttling?

Limit request frequency.

48 ☐ What is Node REPL?

Interactive shell.

49 ☐ What is LTS version?

Long Term Support version.

50 ☐ When should NOT use Node.js?

- Heavy CPU-bound tasks
- Real-time computation-heavy systems

Better to use:

- Go
- Python multiprocessing
- Worker threads